

연산자 우선순위와 괄호 사용법

1. 연산자 우선순위

1.1 연산자 우선순위 개념

- 연산자 우선순위는 수식 내에서 어떤 연산을 먼저 수행할 것인지 결정하는 규칙이다.
- 우선순위가 같다면 좌에서 우로 평가하는 것이 일반적이다.

1.2 연산자 우선순위 순서

1. 괄호 (): 가장 높은 우선순위
2. 단항 연산자: ++ -- 등
3. 산술 연산자: * / % + -
4. 비교 연산자: < > <= >= == !=
5. 논리 연산자: && ||
6. 대입 연산자: = += -= *= /=

1.3 예제 코드

```
#include <stdio.h>

int main() {
    int result1 = 10 + 2 * 3; // 곱셈이 덧셈보다 우선됨
    int result2 = (10 + 2) * 3; // 괄호로 우선순위 변경

    printf("result1: %d\n", result1); // 16
    printf("result2: %d\n", result2); // 36
    return 0;
}
```

2. 괄호 사용 규칙

2.1 괄호 종류

괄호는 크게 세 가지 종류로 구분된다.

1. 소괄호 (): 연산의 우선순위를 정하거나 함수 호출에 사용
2. 중괄호 {}: 코드 블록을 감싸는 데 사용
3. 대괄호 []: 배열 요소 접근에 사용

2.2 괄호 사용 규칙

- 왼쪽 괄호와 오른쪽 괄호의 개수는 일치해야 한다.
- 같은 유형의 괄호에서는 왼쪽 괄호가 먼저 나오고, 오른쪽 괄호가 나중에 나온다.
- 괄호는 중첩될 수 있지만, 포함 관계를 정확히 유지해야 한다.

2.3 잘못된 괄호 사용 예

```
// 잘못된 예제
int result = (3 + 2; // 소괄호가 닫히지 않음
if (a > b] {         // 괄호 종류 불일치
    printf("Error");
}
```

3. 괄호 검사 알고리즘

3.1 괄호 검사 개념

- 문자열 내에서 **괄호의 짝이 맞는지 확인**하는 알고리즘을 작성할 수 있다.
- 일반적으로 **스택(Stack)** 구조를 활용하여 괄호의 균형을 검사한다.

3.2 알고리즘 개요

- 문자열을 왼쪽에서 오른쪽으로 읽어간다.
- 왼쪽 괄호**를 만나면 스택에 삽입한다 (**Push**).
- 오른쪽 괄호**를 만나면 스택에서 하나 제거 (**Pop**)하여 짝이 맞는지 확인한다.
- 문자열을 끝까지 확인한 후 스택이 비어 있으면 괄호가 정상적으로 맞춰진 것이다.

3.3 괄호 검사 예제 코드

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char c) {
    if (top < MAX - 1) {
        stack[++top] = c;
    }
}

char pop() {
    if (top >= 0) {
        return stack[top--];
    }
    return '\0';
}

bool isBalanced(char *expr) {
    for (int i = 0; i < strlen(expr); i++) {
```

```

        if (expr[i] == '(') {
            push(expr[i]);
        } else if (expr[i] == ')') {
            if (pop() != '(') {
                return false;
            }
        }
    }
    return top == -1;
}

int main() {
    char expr[] = "(3 + (2 * 5))";
    if (isBalanced(expr)) {
        printf("괄호가 올바릅니다.\n");
    } else {
        printf("괄호 오류!\n");
    }
    return 0;
}

```

4. 수식 표기법

4.1 중위, 전위, 후위 표기법

1. **중위 표기법 (Infix Notation)**: 우리가 일반적으로 사용하는 방식 (예: $A + B$)
2. **전위 표기법 (Prefix Notation)**: 연산자를 앞에 배치 (예: $+ A B$)
3. **후위 표기법 (Postfix Notation)**: 연산자를 뒤에 배치 (예: $A B +$)

4.2 변환 예제

중위 표기식	전위 표기식	후위 표기식
$2 + 3 * 4$	$+ 2 * 3 4$	$2 3 4 * +$
$(1 + 2) * 3$	$* + 1 2 3$	$1 2 + 3 *$

4.3 중위 -> 후위 변환 알고리즘

- 연산자 우선순위를 고려하여 연산자를 스택에 저장 후 출력

```

#include <stdio.h>
#include <ctype.h>
#include <string.h>

#define MAX 100

char stack[MAX];
int top = -1;

```

```

void push(char c) {
    stack[++top] = c;
}

char pop() {
    return (top >= 0) ? stack[top--] : '\0';
}

int precedence(char c) {
    if (c == '+' || c == '-') return 1;
    if (c == '*' || c == '/') return 2;
    return 0;
}

void infixToPostfix(char* expr) {
    for (int i = 0; i < strlen(expr); i++) {
        if (isalnum(expr[i])) {
            printf("%c", expr[i]);
        } else if (expr[i] == '(') {
            push(expr[i]);
        } else if (expr[i] == ')') {
            while (top != -1 && stack[top] != '(') {
                printf("%c", pop());
            }
            pop();
        } else {
            while (top != -1 && precedence(stack[top]) >= precedence(expr[i])) {
                printf("%c", pop());
            }
            push(expr[i]);
        }
    }
    while (top != -1) {
        printf("%c", pop());
    }
    printf("\n");
}

int main() {
    char expr[] = "(1+2)*3";
    infixToPostfix(expr);
    return 0;
}

```

5. 마무리

- 연산자 우선순위는 괄호를 활용하여 명확하게 정의해야 한다.
- 괄호 검사는 스택을 활용하여 수행 가능하다.
- 중위 표기식을 전위 및 후위 표기식으로 변환하는 알고리즘을 이해해야 한다.

실습을 통해 개념을 확실히 익히는 것이 중요!

